Database Management Systems (DBMS): Knowledge Base

Relational theory, transactions, indexing, query optimization, and distributed database architectures for RAG benchmarking.

RAG Corpus ID: KB-DBMS-02

1. RELATIONAL MODEL & TRANSACTIONS

Relational Algebra & Normalization

Relational Algebra: Selection (σ), Projection (π), Cartesian Product (×), Join (⋈), Set Difference (−), Union (∪).

- **Functional Dependencies (FD):** X → Y (X uniquely determines Y). Armstrong's Axioms: Reflexivity, Augmentation, Transitivity.
- **1NF:** Atomic attribute values, no repeating groups.
- **2NF:** 1NF + No partial dependency (every non-prime attribute fully functionally dependent on whole primary key).
- **3NF:** 2NF + No transitive dependency (non-prime → non-prime). Condition: for X → A, X is superkey or A is prime attribute.
- **BCNF:** For every non-trivial FD X → A, X must be a superkey.

ACID Properties & Recovery

- **Atomicity:** All-or-nothing execution via Undo logging / Write-Ahead Logging (WAL).
- **Consistency:** Invariants & integrity constraints preserved across transactions.
- **Isolation:** Concurrent transactions execute without mutual interference.
- **Durability:** Committed updates survive system crashes via Redo logging & non-volatile storage flushing.
- **ARIES Recovery:** (1) Analysis (reconstruct dirty pages & active txns), (2) Redo (replays logged actions forward), (3) Undo (rolls back uncommitted txns backward).

Concurrency Control & Isolation Levels

Read Phenomena: Dirty Read (reads uncommitted data), Non-repeatable Read (re-read data altered), Phantom Read (new rows inserted).

- **ANSI Isolation Levels:** Read Uncommitted → Read Committed → Repeatable Read → Serializable.
- **Two-Phase Locking (2PL):** Growing Phase (acquire locks) & Shrinking Phase (release locks). Strict 2PL prevents cascading rollbacks by holding exclusive locks until commit.
- **MVCC:** Multi-Version Concurrency Control maintains point-in-time snapshots to enable non-blocking reads during write operations.

Isolation Level	Dirty Read	Non-Repeatable	Phantom
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

2. STORAGE, INDEXING & DISTRIBUTED DB

Storage Structures & Indexing

- **B+ Tree:** Self-balancing search tree where internal nodes store search keys & pointers; leaves hold actual data/pointers linked sequentially for rapid range queries. Balanced depth O(log n).
- **LSM-Tree (Log-Structured Merge):** Optimizes write throughput by appending writes to MemTable (RAM) and WAL, later flushing sequentially to immutable SSTables (disk) with background compaction.
- **Clustered vs Non-Clustered:** Clustered index physically sorts data records on disk (one per table); Non-clustered index creates separate pointer table.

Query Processing & Optimization

- **Evaluation Pipeline:** SQL Query → Parser/Lexer → Logical Plan (Relational Algebra Tree) → Cost-Based Optimizer (CBO) → Physical Execution Plan.
- **Join Algorithms:** Nested Loop Join (O(M×N)), Block Nested Loop, Index Nested Loop, Hash Join (build hash table on smaller relation, probe with larger), Sort-Merge Join (requires presorted input).

Distributed Databases & Consensus

- **CAP Theorem:** A distributed system can guarantee at most two of: Consistency (linearizability), Availability (every request gets non-error response), Partition Tolerance (tolerates network drops).
- **Two-Phase Commit (2PC):** Phase 1 (Prepare / Vote: Coordinator requests all participants to prepare), Phase 2 (Commit / Abort: Execute based on unanimous vote). Vulnerable to coordinator crash.
- **Distributed Consensus:** Paxos and Raft (Leader Election, Log Replication, Safety Invariants) ensure linearizable replicated state machines.
- **Sharding / Partitioning:** Horizontal data distribution via Range Sharding (risk of hot spots) or Consistent Hashing (uniform ring distribution).

Index / System	Type	Optimized Workload
B+ Tree	Disk Page Tree	Read-heavy OLTP, Range Scans
LSM-Tree	Append / SSTable	Write-heavy, Time-series, NoSQL
Raft / Paxos	Consensus Protocol	Distributed fault-tolerant logs

RAG Validation Note: Designed for dense semantic search, entity linking, SQL vs NoSQL comparisons, transaction isolation checks, and consensus retrieval tests.